UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos (11[0|2]76)

Clase 05 • **Ejercicio 01** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 1 (Live Coding): Billetera Virtual y Saldos Multidivisa con TDD

Contexto

En este ejercicio consolidaremos el uso fluido del ciclo **Test-Driven Development (TDD)** (*Red - Green - Refactor*) para modelar y blindar invariantes de negocio estrictas en memoria (saldos no negativos, precondiciones sobre montos y unicidad de cuentas por divisa), respetando la regla **"Tell, Don't Ask"**.

Enunciado

Una **Billetera Virtual** (`Billetera`) administra un conjunto de cuentas bancarias (`Cuenta`), cada una asociada a una divisa específica (ej. `"ARS"` , `"USD"` , `"EUR"`).

Reglas de negocio e invariantes:

1. **Apertura de Cuentas:** Toda cuenta se inicia con saldo `0.0` .
2. **Unicidad de Divisa:** Un usuario no puede poseer dos cuentas para la misma divisa dentro de su billetera. Intentar abrir una divisa ya existente debe lanzar `CuentaDuplicadaException` (la verificación debe ser insensible a mayúsculas o espacios accidentales, ej. `"usd"` vs `"USD"`).

3. **Precondición de Montos:** Toda operación de depósito o extracción exige un monto estrictamente positivo (> 0). Si se recibe un monto ≤ 0, debe lanzarse `IllegalArgumentException`.
4. **Saldo No Negativo (Invariante Estricta):** El saldo de una cuenta nunca puede quedar en negativo. Si se intenta extraer más dinero del disponible, se debe abortar la operación lanzando `SaldoInsuficienteException` y **garantizar que el saldo previo permanezca inalterado**.
5. **Divisas Habilitadas:** Cualquier operación (`depositar`, `extraer`, `getSaldo`) sobre una divisa no registrada previamente en la billetera debe rechazarse con `IllegalArgumentException`.

Tareas:

1. Utilice TDD guiando el diseño de las clases `Cuenta`, `Billetera` y las excepciones de dominio `SaldoInsuficienteException` y `CuentaDuplicadaException`.
2. Escriba pruebas unitarias con JUnit 5 avanzando incrementalmente desde los casos elementales (saldo inicial, depósitos) hasta las invariantes estrictas (descubierto, unicidad y validación de argumentos).
3. Asegúrese de que `Billetera` ofrezca las operaciones públicas:
    - `abrirCuenta(String divisa)`: Registra la cuenta validando no duplicidad.
    - `depositar(String divisa, double monto)`: Acredita el monto delegando en la cuenta.
    - `extraer(String divisa, double monto)`: Debita el monto delegando en la cuenta.
    - `getSaldo(String divisa)`: Retorna el saldo actual de la cuenta solicitada.

Restricciones y Tips

- **Regla "Tell, Don't Ask":** La clase `Billetera` no debe inspeccionar el saldo de `Cuenta` para decidir si alcanza (`if (cuenta.getSaldo() >= monto)`). `Billetera` debe limitarse a delegar la orden `cuenta.extraer(monto)`, dejando que `Cuenta` valide y proteja su propia invariante.

- **Testing de Excepciones y Atomicidad de Estado:** Al utilizar `assertThrows` para validar que una operación rechazada lanza la excepción esperada, verifique siempre a continuación que el estado del objeto no haya sufrido alteraciones colaterales.
- **Fase de Refactor:** Aproveche la suite de pruebas como red de seguridad para normalizar las divisas con `.trim().toUpperCase()` y para encapsular la búsqueda interna de cuentas.

Extensión Complementaria (Opcional - Bonus Track): Transferencias y Objetos Inmutables

Si concluye las tareas básicas, extienda el sistema implementando transferencias monodivisa entre billeteras:

1. Diseñe la clase inmutable `Transferencia` como comprobante de transacción, conteniendo: `divisa`, `monto`, `fechaHora` (`LocalDateTime`) y un texto de `detalle`.
    - **Inmutabilidad estricta:** Una vez instanciada, ninguna de sus propiedades puede alterarse (atributos `final`, ausencia total de *setters*).
2. Agregue en `Billetera` la operación:
    - `transferir(Billetera destino, String divisa, double monto) : Transferencia`
3. **Garantía de Atomicidad:** Si el origen no tiene fondos suficientes, debe abortar la operación con `SaldoInsuficienteException`, asegurando que **ninguna de las dos billeteras muta su saldo** y que no se emite comprobante alguno.
4. **Precondiciones del Destino:** Si la billetera destino es `null` o es la misma billetera origen, la operación debe rechazarse con `IllegalArgumentException`.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Por qué es superior delegar la validación del saldo dentro de `Cuenta.extraer(monto)` en lugar de que la `Billetera` verifique `if (cuenta.getSaldo() >= monto)` antes de llamar a la extracción?
2. ¿Por qué en TDD, al testear el lanzamiento de una excepción por violación de invariantes, es indispensable asertar también el estado posterior del objeto?
3. ¿Cómo evita la defensa rigurosa de invariantes de estado que la aplicación entre en estados inconsistentes e inmanejables en memoria?
4. ¿Qué ventajas arquitectónicas y de diseño aporta modelar `Transferencia` como una clase inmutable frente a una entidad tradicional con getters y setters?
5. ¿Cómo se proyectan los patrones de diseño e invariantes aprendidos en esta billetera hacia los Ejercicios 2 (unicidad de tarjetas) y 3 (mercado y cotizaciones dinámicas)?